

ISIT307 - WEB SERVER PROGRAMMING

LECTURE 1.1 - INTRODUCTION



LECTURER/COORDINATOR

Lecturer

- Dr. Elena Vlahu-Gjorgievska
 - e-mail: elenavg@uow.edu.au

Tutor

- Poh Kok Loo
 - E-mail: lubg9@outlook.com

LECTURE PLAN

- What you need to know about lectures, labs, assignments and exams
- Introduction to PHP

SUBJECT STRUCTURE

- Lectures
- Labs
- Assignments : 2 assignments

ASSESSMENT

Assessment Items	Percentage of Final Mark		Due Date
	Marks for the Item	Minimum required for a pass	
Assignment 1 (group work – 2 students)	25	N/A	As scheduled
Assignment 2 (individual work)	25		As scheduled
Final Exam (TBA)	50	20	Exam week as per schedule
Total	100	50	The mark must be ≥ 50 to pass the subject

LECTURES

- The lectures will introduce fundamental concepts and the principles of web server programming.
- The lectures will contain a sufficient number of examples to facilitate explanation of complex technical aspects.
- It is highly recommended that you implement all examples, compile and run the programs on your computer.
- I'm encouraging you to actively participate in the lecture sessions answering questions and making your own notes that will help you to better understand the material.

TUTORIALS

- Students are expected to complete the tasks during a supervised tutorial session. If more time is required to complete all exercises, this can be done before or after the tutorials.
- During the scheduled tutorials, the assignments 1 and 2 need to be presented to the tutor and short Moodle quizzes needs to be completed.

ADDITIONAL MATERIALS

- Additional materials (websites, readings and videos) can be used.
- It is a good practice for you to read/watch/implement the examples from these materials.

ASSIGNMENTS

- There will be two assignments.
- When an assignment is released, download the assignment description from the subject web site. Read carefully the specifications. Make sure you understand the requirements.
- Your solutions must be submitted electronically via the subject web site (Moodle). No submission via email will be accepted.
- Late assignments will not be accepted without a granted academic consideration.
- During the scheduled tutorials, the assignments 1 and 2 need to be presented to the tutor and short Moodle quizzes needs to be completed. The tutor will assess your solution and give you a mark according to the quality of your solution and the level of your understanding.

ASSIGNMENTS

- When you submit an assessment task, you are declaring the following:
 - It is your own work and you have not copied anything from others and you have not discussed your work with others.
 - You have not plagiarised from published work (including various internet sources).
 - You have read your responsibilities under the UOW's policy on plagiarism and you understand possible consequences.
 - You have not used storage devices which can be accessed by others without passwords.
- Plagiarism = Big problems
- You may be asked to have a formal meeting with the lecturer to explain your assignment solution if there are doubts that you worked on your assignment yourself.

SUBJECT WEB SITE

- All important notices related to the subjects will be posted on the subject's Moodle site.
- Check it frequently!
- **Note: For any information published on the subject's website, it is considered that all students have been notified!**

SELF-DIRECTED STUDY

- **Listening passively is useless!**
- Attend all lectures. Take your own notes and add your own comments or questions during the lectures.
- Read/implement all materials/solutions posted on the subject web site.
- Implement examples discussed at lectures or developed during labs.
- If you have any questions, discuss it with me.

SUBJECT MATERIALS

- **Recommended books:**

- Sommerfeld, R., 2024. Unlock PHP 8: From Basic to Advanced. BPB Publications (Available in UOW Library as ebook)
- Gosselin, D., Kokoska, D. and Easterbrooks, R., 2011. PHP Programming with MySQL - The Web Technologies Series (2nd edition). Cengage Learning.
- White III, E. and Eisenhamer, J.D., 2007. PHP 5 in Practice. Pearson Education. (Available in UOW Library as ebook)
- Nixon, R., 2021. Learning PHP, MySQL, and JavaScript : a step-by-step guide to creating dynamic websites, 6th edition. O'Reilly Media Incorporated. (Available in UOW Library as ebook)
- Connolly, R., 2015. Fundamentals of web development. Pearson Education. (Available in UOW Library as ebook)
- Rahman, M., 2017. PHP 7 Data Structures and Algorithms. Packt Publishing Ltd.
- Nixon, R., Comer, R., Demarest, R., 2015. Learning PHP, MySQL & JavaScript : with jQuery, CSS & HTML5 , Fourth edition., O'Reilly Media Inc. (Available in UOW Library as ebook)

- **Lecture notes & Labs:**

- The lecture notes are available on the subject web site (The lecture notes may not include some examples and explanations given in lectures).
- The Labs exercises are available on the subject web site.

- **Additional materials**

- <http://www.w3schools.com>
- <http://php.net/>
- Additional materials may be posted on the subject web site.

SOFTWARE REQUIREMENTS

- Notepad++ (Windows); Sublime Text, BBEdit, TextMate (Mac)
- WAMP Server (it comes with)
 - Apache server
 - PHP v.8.1+
 - MySQL
- Instead of WAMP can be used MAMP, LAMP or XAMPP

OBJECTIVES

- PHP syntax and basic structure
- Variables and constants
- Data types
- Operators and expressions

PHP

- PHP (PHP: Hypertext Preprocessor) is a server scripting language, and a powerful tool for making dynamic and interactive Web pages
- PHP is a widely-used open source scripting language (free)
- PHP scripts are executed on the server
- PHP runs on various platforms (Windows, Linux, MacOS, etc.) and is compatible with almost all servers used today (Apache, IIS, etc.)
- PHP supports a wide range of databases
- PHP runs efficiently on the server side

PHP

- **Embedded language** refers to code that can be embedded within a Web page (HTML document)
- PHP files can contain text, HTML, CSS, JavaScript, and PHP code
- PHP code is typed into a separate section
- A script containing PHP code must be saved with an extension of .php
- PHP code are executed on the server, and the result is returned to the browser as plain HTML
- PHP code is never sent to a client's Web browser; only the output of the processing is sent to the browser

BASIC STRUCTURE OF A PHP SCRIPT

- **The PHP code** is written between PHP script delimiters
 - **<?php** - opening tag
 - **?>** - closing tag
- A PHP script can contain multiple statements
 - each statement must end with a semicolon (;)

ECHO AND PRINT

- The `echo` and `print` are **language constructs** (built-in features of a programming language) that produce output
 - used to return result/output from the PHP script
- The text passed to `echo` is called a “literal string” and must be enclosed in either single or double quotation marks
- `echo` can take multiple arguments (separated with comma) and does not return a value
- `print` can take only 1 argument and returns a value of 1 if successful or a value of 0 if not successful

EXAMPLE

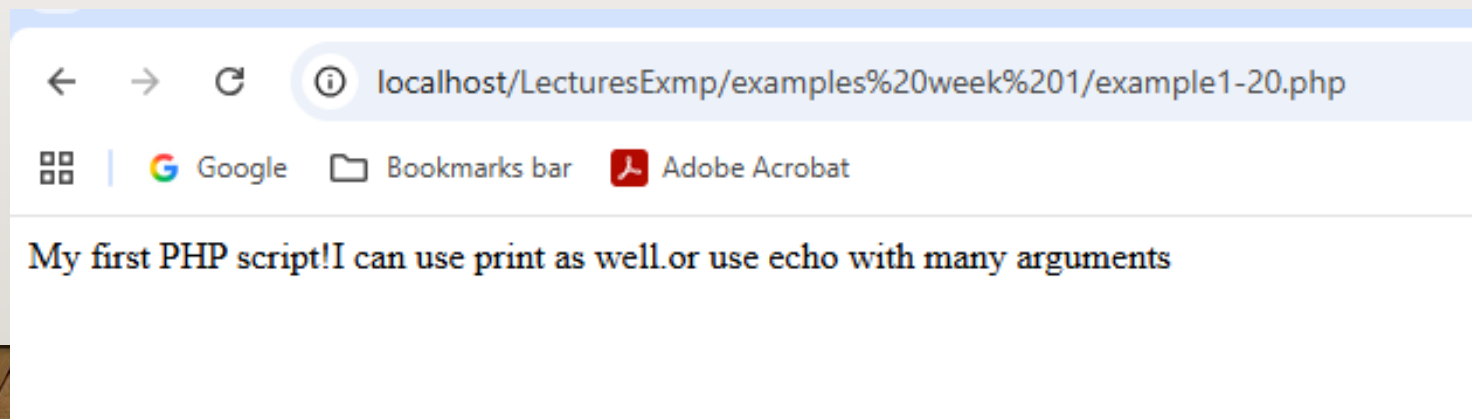
```
<?php
```

```
    echo "My first PHP script!";
```

```
    print "I can use print as well.";
```

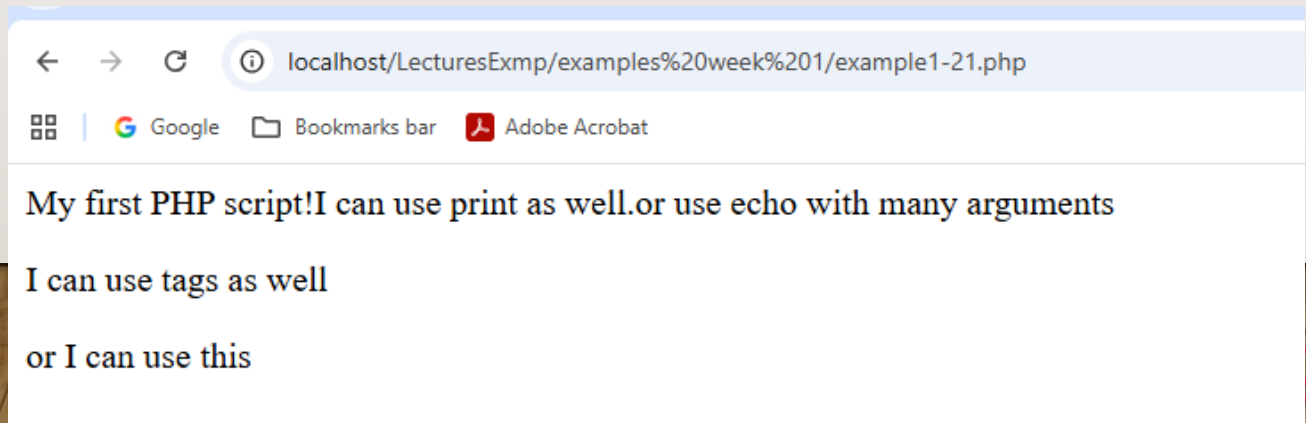
```
    echo "or use echo", " with many ", "arguments";
```

```
?>
```



EXAMPLE

```
<html>
<head><title>PHP Code</title></head>
<body>
<p>
<?php
    echo "My first PHP script!";
    print "I can use print as well.";
    echo "or use echo", " with many ", "arguments";
?>
</p>
<?php echo "<p>I can use tags as well</p>"; ?>
<?= "or I can use this" ?></body>
</html>
```



CASE SENSITIVITY IN PHP

- PHP language constructs are mostly case **insensitive**

```
<?php
```

```
echo "<p>Explore <strong>Africa</strong>, <br />";
```

```
Echo "<strong>South America</strong>, <br />";
```

```
ECHO " and <strong>Australia</strong>!</p>";
```

```
?>
```

- Variables and constant names are case sensitive

COMMENTS IN PHP SCRIPT

- **Comments** are not executed, but are used to provide helpful information, such as:
 - notes related to/explaining the code
 - instructions to programmers who might need to modify the code
- **Single-line comments**
 - Add `//` or `#` before the text
- **Multi-line / Block comments**
 - Add `/*` at the beginning
 - And `*/` at the end

VARIABLES AND CONSTANTS

- **Variables** are used to store and manipulate information/data
- The values, or data, contained in variables are classified as **data types**
- Rules and conventions
 - variable must start with a dollar sign (\$), followed by the name of the variable
 - variable name must start with a letter or underscore and may contain uppercase and lowercase letters, numbers, or underscores
 - variable name cannot contain spaces
- Variable names are case sensitive

VARIABLES

- Declare and initialize a variable

```
$variable_name = value;
```

- Display a variable's value

```
$Num = 18;
```

```
echo $Num;
```

```
echo "<p>My number is ", $Num, ".</p>";
```

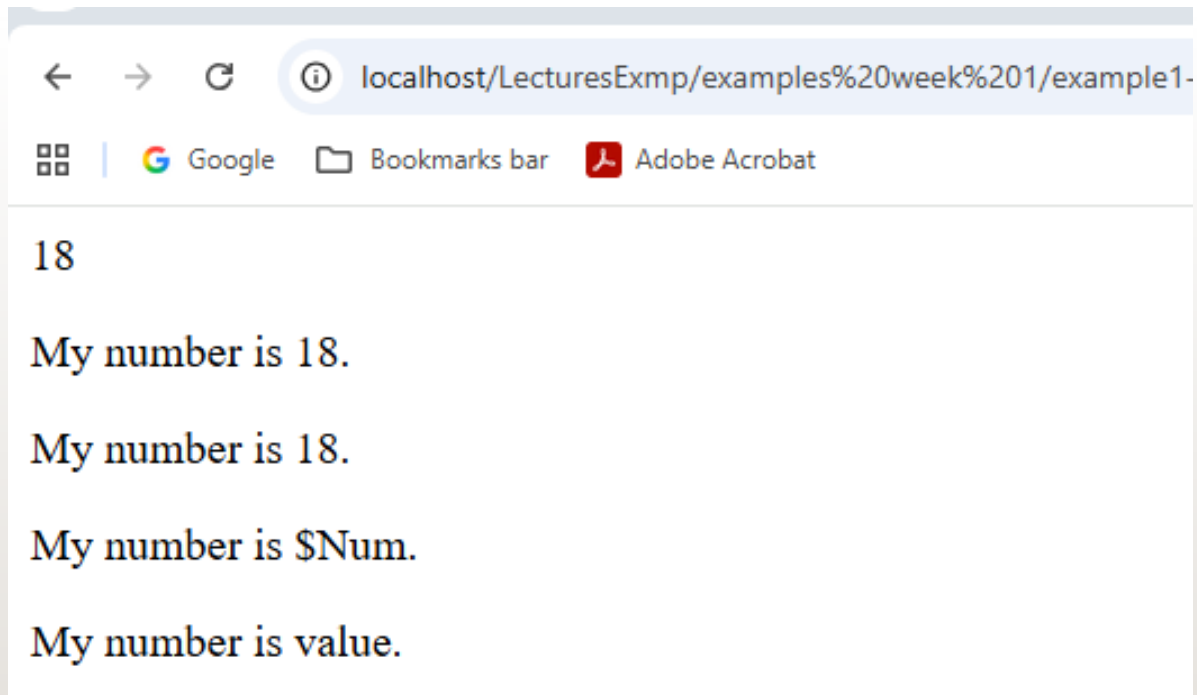
```
echo "<p>My number is $Num.</p>";
```

```
echo '<p>My number is $Num.</p>';
```

```
$Num = "value";
```

```
echo "<p>My number is $Num.</p>";
```

VARIABLES



CONSTANTS

- A **constant** contains information/data that does not change during the course of program execution
- **define ()** function can be used to create a constant

```
define("CONSTANT_NAME", value);
```

- **const** keyword can be used to declare constant at the top-level scope (it cannot be used to declare constant inside functions, loops, if/else statements or try/catch blocks)

```
const CONSTANT_NAME = value;
```

- Unlike variables, constant names cannot be included within the quotation marks of the `echo` statement

DATA TYPES

- The PHP language supports:
 - **string, int, float, bool, null**
 - **Reference** or **composite** data types, which contain multiple values or complex types of information
 - Reference data types: **arrays** and **objects**
 - “**resource**” data type that holds a reference to an external resource (e.g. XML file)

DATA TYPES

- **Strongly typed programming languages** require you to declare the data types of variables
- **Static or strong typing** refers to data types of the variables that do not change after they have been declared
- **Loosely typed programming languages** do not require you to declare the data types of variables
- **Dynamic or loose typing** refers to data types of the variables that can change after they have been declared

ARRAYS

- An **array** contains a set of data represented by a single variable name
- An **element** refers to each piece of data that is stored within an array
- *In PHP the values assigned to different elements with same array can be of different types
- An **index** is an element's numeric position within the array
 - By default, indexes begin with the number zero (0)
 - An element is referenced by enclosing its index in brackets at the end of the array name:

```
$myArray[1]
```

ARRAYS

- The `array()` construct syntax is:

```
$array_name = array(values) ;
```

```
$myArray = array(  
    "black",  
    "white",  
    "green",  
    "red",  
    "yellow") ;
```

ARRAYS

- Array name and brackets syntax is:

\$array_name[]

```
$myArray[] = "black";  
$myArray[] = "white";  
$myArray[] = "green";  
$myArray[] = "red";  
$myArray[] = "yellow";
```


ARRAYS

- The `print_r()` function displays the index and value of each element in an array
- The `var_dump()` function displays the index, value, data type and number of characters in the value
- The `var_export()` function is similar to `var_dump()` function

ARRAYS

- modify an array element

```
$myArray = array(  
    "black",           // first element(0)  
    "white",           // second element(1)  
    "green");          // third element(2)
```

```
$myArray[0] = "yellow";
```

AVOIDING ASSIGNMENT NOTATION PITFALLS

- Assigns the string “Hello” to a variable named \$list

```
$list = "Hello";
```

- Assigns the string “Hello” to a new element appended to the end of the \$list array

```
$list[] = "Hello";
```

- Replaces the value stored in the first element (index 0) of the \$list array with the string “Hello”

```
$list[0] = "Hello";
```

TYPE CONVERSION

- **Type conversion** or **type casting** converts the value contained in a variable of one data type into another data type
- The PHP syntax for casting is:

```
$NewVariable = (new_type) $OldVariable;
```
- *(new_type)* refers to the type-casting operator representing the type to which you want to convert
- PHP can convert string into numeric value if the string starts with numeric value, any subsequent non-numeric characters are ignored

TYPE CONVERSION

- **Type conversion** can also be done using type conversion functions
 - `intval()`,
 - `floatval()`,
 - `strval()`
- `gettype()` function returns the data type of the variable
- `is_*()` functions
 - `is_numeric($a)`, `is_int($a)`, `is_string($a)`

OPERATORS AND EXPRESSIONS

- **Operators** are symbols (+, *) that represent operations
- An **expression** is a literal value or variable (or a combination of literal values, variables, operators or other expressions) that can produce a result
- A **literal** is a static value such as a literal string or a number

OPERATORS AND EXPRESSIONS

- A **binary operator** requires an operand before and after the operator
 - `$MyNumber = 100;`
- A **unary operator** requires a single operand either before or after the operator

ARITHMETIC OPERATORS

- **Arithmetic operators** are used in PHP to perform mathematical calculations

Symbol	Operation	Description
+	Addition	Add two operands
-	Subtraction	Subtracts the right from the left operand
*	Multiplication	Multiplies two operands
/	Division	Divides the left by the right operand
%	Modulus	Divides the left by the right operand and returns the remainder
**	Exponentiation	Left the left operand to the power of the right operand

ARITHMETIC OPERATORS

- The increment (++) and decrement (--) unary operators increases/decreases the value of the operand by 1
- Can be used as prefix or postfix operators
 - A **prefix operator** is placed before a variable
 - A **postfix operator** is placed after a variable

ASSIGNMENT OPERATORS

- **Assignment operators** are used for assigning a value to a variable:

```
$myNumber = 123;
```

```
$myText = "PHP";
```

- **Compound assignment operators** perform mathematical calculations on variables and literal values in an expression, and then assign the calculated/new value to the left operand

```
$a += $b;
```

ASSIGNMENT OPERATORS

Symbol	Operation	Description
=	Assignment	Assigns the value of the right operand to the left operand
+=	Compound addition assignment	Adds the value of the right operand to the value of the left operand and assigns the new value to the left operand
-=	Compound subtraction assignment	Subtracts the value of the right operand from the value of the left operand and assigns the new value to the left operand
*=	Compound multiplication assignment	Multiplies the value of the right operand by the value of the left operand and assigns the new value to the left operand
/=	Compound division assignment	Divides the value of the left operand by the value of the right operand and assigns the new value to the left operand
%=	Compound modulus assignment	Divides the value of the left operand by the value of the right operand and assigns the remainder (modulus) to the left operand

COMPARISON OPERATORS

- **Comparison operators** are used to compare two operands returning a boolean value of `TRUE` or `FALSE`
- The comparison operator *compares* values
- Comparison operators are used with **conditional statements** and **looping statements**

COMPARISON OPERATORS

Symbol	Operation	Description
==	Equals	Returns TRUE if the operands are equal
===	Identical/Strict equal	Returns TRUE if the operands are equal and of the same data type
!=	Not Equal	Returns TRUE if the operands are not equal
!==	Not identical	Returns TRUE if the operands are not equal or not of the same data type
<	Less than	Returns TRUE if the left operand is greater than the right operand
>	Greater than	Returns TRUE if the left operand is less than the right operand
<=	Less than or equal to	Returns TRUE if the left operand is greater than or equal to the right operand
>=	Greater than or equal to	Returns TRUE if the left operand is less than or equal to the right operand
<=>	Spaceship operator	Returns an integer (-1/0/1) depending on whether the left operand is less than, equal to, or greater than right operand

LOGICAL OPERATORS

- **Logical operators** are used for comparing two boolean operands for equality, usually used to combine conditional statements

Symbol	Operation	Description
&& AND	And	Returns true if both operands are true
 OR	Or	Returns true if at least one of the operands
xor	XOR	Returns true if exactly one of the operands is true, but not both is true
!	NOT	Inverts the logical value of the operand

CONCATENATION OPERATOR

- **Concatenation operator** joins two or more strings together

```
$txt1 = "I'm learning";  
$txt2 = "PHP";  
$txt = $txt1.$txt2;  
echo " <p> $txt </p>";
```

```
$txt = "I'm learning";  
$txt .= "PHP";  
echo " <p> $txt </p>";
```

OPERATOR PRECEDENCE

- **Operator precedence** refers to the order in which operations in an expression are evaluated
- **Associativity** is the order in which operators of equal precedence execute
 - Associativity is evaluated on a left-to-right or a right-to-left basis

OPERATOR PRECEDENCE

Symbol	Operator	Associativity
clone new	clone and new	(n/a)
**	arithmetic	right
+ - ++ --- ~ (int) (float) (string) (array) (object) (bool) @	arithmetic, type casting and error control	(n/a)
<u>instanceof</u>	type	left
!	logical	(n/a)
* / %	arithmetic	left
+ - .	<u>arithmetic</u> , array and string (. prior to PHP 8.0.0)	left
<< >>	bitwise	left
.	string (as of PHP 8.0.0)	left
>	pipe	left
< <= > >=	comparison	non-associative
== != === !== <> <=>	comparison	non-associative
&	bitwise and references	left
^	bitwise	left
	bitwise	left
&&	logical	left
	logical	left
??	null coalescing	right
? :	ternary (left-associative prior to PHP 8.0.0)	non-associative
= += -= *= **= /= .= %=	assignment	right
&= = ^= <<= >>= ??=		
yield from	yield from	(n/a)
yield	yield	(n/a)
print	print	(n/a)
and	logical	left
<u>xor</u>	logical	left
or	logical	left
throw	throw	(n/a)

